1. Cesar Crusius' Literate Programming Toolkit. I wrote these tools in order to have a better way of writing my TEX macros, specifically those that involve a lot of Lua code. The `WEB` and `CWEB` tools were not language agnostic, and other tools either had no Windows binaries available, or did not generate plain TEX documentation. Since writing simple literate programming weavers and tanglers is not too hard (the weaver and tangler were both pretty functional at less than 200 lines each), I went ahead and wrote my own.

2. The `CCWEB` syntax mostly a (small) subset to that of `CWEB`, but there are some differences that make it incompatible, less complicated, and more versatile:

- Sections start with either "`@␣`" or "`@*`", when they appear at the beginning of a line. Just as with `CWEB`, the latter starts a "main section," which is typeset with a bold title, on a new page.
- Section names are specified with "`@<`*section name*`@>`," and code parts start with "`@<`*section name*`@>=`." (In `CWEB` those start "C parts," but `CCWEB` is language agnostic.) Differently from `CWEB`, `CCWEB` allows any string to appear after a "`@<`*section name*`@>=`," and completely ignores it. This allows one to signal other tools the language that follows, or anything else one wishes. The `ccweb.ccw` file, for example, marks Lua code with the string `{lua}`, TEX code with the string `{tex}`, and my Vim setup then correctly syntax highlights those sections as Lua and TEX code.
- Verbatim text is enclosed within "`|`", as long as both delimiters occur on the same line.
- The quoting character is "`@`," that is, apart from the commands above, `@`x writes the character x to both the weaved and tangled outputs. In `ccweb.ccw`, for example, the string "`@`x" is typed as `|@@|\@|x` (the `cwebmac` `\|#1` macro typesets `#1` as an identifier), and that in turn as `|@|@@@@@|\@@@|x|`. The reader can figure it out for himself how is *that* typed.

3. Gone is the `@c` command that specifies the main section. Instead, `CCWEB` adopts a naming convention: when `cctangle` is run, the contents of the `@<*@>` section are written to `stdout`, while the contents of sections named `@<*{`*file name*`}@>` are written to the specified file name. The `ccweb.ccw` file uses this functionality to create both `ccweave.lua` and `cctangle.lua` with only one run of `cctangle`.

4. `CCWEB` uses the `cwebmac` macros when generating TEX documentation, so the output should be as similar as possible to what you are used to.

5. Main sections. There is a reasonable amount of common code shared between `CCWEAVE` and `CCTANGLE`, including the code that reads a `CCWEB` file into suitable data structures, common string processing utilities, and so on.

⟨ Common `ccweb` code 5 ⟩ ≡
⟨ Global variables 27 ⟩
⟨ Functions 9 ⟩
⟨ Parsing functions 16 ⟩

This code is used in sections 6 and 7.

6. ⟨ `*{cctangle.lua}` 6 ⟩ ≡
⟨ Common `ccweb` code 5 ⟩
⟨ Tangle file 34 ⟩
⟨ Main `CCTANGLE` procedure 37 ⟩

7. ⟨ `*{ccweave.lua}` 7 ⟩ ≡
⟨ Common `ccweb` code 5 ⟩
⟨ `CCWEAVE` global variables 40 ⟩
⟨ `CCWEAVE` functions 42 ⟩
⟨ Main `CCWEAVE` procedure 58 ⟩

8. Utility functions. These are in two categories: string manipulation and debugging.

9. Lua lacks a basic `str[i]` construction to get a character from a string. We fix the problem with a `string_at` function.

⟨ Functions 9 ⟩ ≡

```
local function string_at(str,idx)
  if not str or not idx then return nil end
  if string.len(str) < idx then return nil end
  return string.sub(str,idx,idx)
end
```

See also sections 10, 11, and 12.

This code is used in section 5.

10. We also have the need to check whether a string starts with a certain expression or not. The `starts_with` function does that, accepting both a string and a list of strings.

⟨ Functions 9 ⟩ +≡

```
local function starts_with(line,expr)
  if not line then return false end
  if type(expr) == "string" then
    return (string.find(line,"^"..expr))
  elseif type(expr) == "table" then
    for k,v in pairs(expr) do
      if starts_with(line,v) then return true end
    end
  else
    assert(false,"starts_with can not handle '"..type(expr).."' expression.")
  end
  return false
end
```

11. When printing both code and TEX documentation, we have to process the `@` quotes, and we do it all at one go by calling the `unquoteline` function.

⟨ Functions 9 ⟩ +≡

```
local function unquoteline(line)
  return (string.gsub(line,"@\(.\)","%1"))
end
```

12. Finally, once I started fancyfying this code, I started running into many problems. Since Lua does not have support for C-style `#line` directives, I chose to insert debug output manually into the code, until I found the problem.

Apart from `io.stderr:write`, there is a need to print data structures, which are Lua tables. The function below takes care of that.

⟨ Functions 9 ⟩ +≡

```
local function print_table(tree,file,prefix)
  file:write("{\n")
  for k,v in pairs(tree) do
    file:write(prefix.."  ["..tostring(k)..."] = ")
    if type(v) == "table" then print_table(v,file,prefix.."    ")
    else file:write(tostring(v)..."\n") end
  end
  file:write(prefix..."}\n")
end
```

13. Sections. A CCWEB section is a chunk of documentation optionally attached to a named chunk of code. Sections that start with `@*` are "starred."

⟨ Empty section 13 ⟩ ≡

```
{
  ["star"]  = false,
  ["body"]  = {},
  ["name"]  = nil,
  ["code"]  = {},
  ["depth"] = 1
}
```

This code is used in section 22.

14. The body part of a section is a sequence of text lines, which in turn consist of an alternating list of normal text and verbatim text. We represent that list by the original line, plus a list of pairs of indices indicating the start and finish of blocks on that line. For example, if one line reads

```
Function |foo_bar| is not |foo||bar|.
1 2345678901234567890123456789012345 67
          1         2         3
```

then the structure representing it will look like

```
{
  ["line"] = "Function |foo_bar| is not |foo||bar|.",
  ["nodes"] = { {1,9}, {11,17}, {19,26}, {28,30}, false, {33,35}, {37,37} }
}
```

Note that the list of pairs is *always* alternating between normal text and verbatim text, starting with normal text. If the original line does not have an alternation, missing items are signaled with a `false` entry.

⟨ Empty body text 14 ⟩ ≡

```
{ ["line"] = "", ["nodes"] = {} }
```

This code is used in section 17.

15. The code part of a section is similarly a sequence of code lines, which in turn are a sequence of verbatim code text and references to other sections. The references are represented in the same way a body line is, that is, as an alternating sequence of normal and verbatim text. The sequence always starts with a verbatim code text, and any missing elements are represented with a `false` entry. Thus, the code line

```
function foo_bar = @<Function |foo_bar| is not |foo||bar|.@> end
```

is represented as

```
{
  "function foo_bar = ",
  {
    ["line"] = "Function |foo_bar| is not |foo||bar|.",
    ["nodes"] = { {1,9}, {11,17}, {19,26}, {28,30}, false, {33,35}, {37,37} }
  },
  " end"
}
```

16. Basic parsing. The building block of line parsing is the `parse_chunk` function: it processes characters from the given line, one by one, keeping quotes intact, until `EOL` or any expression in the `stop` list is found at the beginning of what remains to be parsed. Once `EOL` or `stop` is found, the function returns what it parsed, and what remains to be parsed.

⟨ Parsing functions 16 ⟩ ≡

```
local function parse_chunk(line,stop)
  local result, rest = "", line
  while rest ~= "" and not starts_with(rest,stop) do
    local c = string_at(rest,1)
    if c == "@" then
      result = result .. c
      rest = string.sub(rest,2)
      c = string_at(rest,1)
      assert(c,"End of line '"..line.."' found while unquoting.")
    end
    result = result .. c
    rest = string.sub(rest,2)
  end
  return result, rest
end
```

See also sections 17, 21, 22, 26, 28, and 31.

This code is used in section 5.

17. Reading section text. Let us now start buildin the section structures from lines of input. The function `parse_text_line` builds a structure from the given line until what remains to be parsed starts with `stop`.

The function goes about its way by processing a pair of normal and verbatim text at every loop run, and is rather simple in spirit – Lua makes the code look a bit noisy, though. The return value is a pair with the text structure, and the part of the line that remains to be parsed.

⟨ Parsing functions 16 ⟩ +≡

```
local function parse_text_line(line,stop)
  local result = ⟨ Empty body text 14 ⟩
  local rest = line
  while rest~="" and not starts_with(rest,stop) do
    local text
    ⟨ Parse normal text 18 ⟩
    ⟨ Parse verbatim text 20 ⟩
  end
  assert(starts_with(rest,stop),"End of line '"..line.."' found while parsing text.")
  return result,rest
end
```

18. When parsing normal text, we must keep track on how did it end – if at the start of verbatim text, or at the specified end expression – and process it accordingly.

⟨ Parse normal text 18 ⟩ ≡

```
text,rest = parse_chunk(rest,{"|",stop})
if text ~= "" then ⟨ Insert node in result["nodes"] 19 ⟩
elseif not starts_with(rest,stop) then table.insert(result["nodes"],false)
end
if starts_with(rest,stop) then return result, rest end
assert(rest~="","End of line '"..line.."' found while parsing text.")
```

This code is used in section 17.

19. ⟨ Insert node in `result["nodes"]` 19 ⟩ ≡

```
table.insert(result["nodes"],{
  string.len(result["line"])+1,
  string.len(result["line"])+string.len(text)
})
result["line"]=result["line"]..text
```

This code is used in sections 18 and 20.

20. To parse verbatim text, we first remove the leading `|` from the string, parse a chunk until we find the matching `|`, and insert the text in such a way that `result["line"]` contains the delimiters, but the index pair points to whats in between them.

⟨ Parse verbatim text 20 ⟩ ≡

```
rest=string.sub(rest,2)
text,rest = parse_chunk(rest,"|")
assert(rest~="","End of line '"..line.."' found while processing verbatim.")

result["line"]=result["line"].."|"
if text ~= "" then ⟨ Insert node in result["nodes"] 19 ⟩
else table.insert(result["nodes"],false) end
result["line"]=result["line"].."|"

rest=string.sub(rest,2)
```

This code is used in section 17.

21. Reading section code. With the `parse_text_line` function out of the way, building an equivalent `parse_code_line` function is relatively easy, as it follows basically the same motions, this time alternating between code and section references. The function gets an input line, and returns a list suitable for the `["code"]` section element.

⟨ Parsing functions 16 ⟩ +≡

```
local function parse_code_line(line)
  local result = {}
  local rest = line
  while rest ~= "" do
    local text

    -- Code
    text, rest = parse_chunk(rest,{"@<","$"})
    if text ~= "" then table.insert(result,text)
    elseif starts_with(rest,"@<") then table.insert(result,false)
    end
    if rest == "" then return result end

    -- Section reference
    rest = string.sub(rest,3)
    text, rest = parse_text_line(rest,"@>")
    assert(rest~="","End of line '"..line.."' found while processing section reference.")
    table.insert(result,text)
    rest = string.sub(rest,3)
  end
  return result
end
```

22. Parsing section definitions. The `get_section` function builds a basic section structure from a given line, and returns this structure and the part of the line that comes after the section command. It the line does not start a section, the function returns `nil`.

⟨ Parsing functions 16 ⟩ +≡
```
local function get_section(line)
  local section = ⟨ Empty section 13 ⟩
  local match
  ⟨ Process @*n section start 23 ⟩
  ⟨ Process @* section start 24 ⟩
  ⟨ Process @␣ section start 25 ⟩
end
```

23. For determining the section depth, `CWEB`'s documentation seems to imply that "`@**`" sections have depth -1, but a quick test run reveals otherwise: "`@**`" sections have depth 0, "`@*`" sections have depth 1, and "`@*`n" sections have depth $n+1$.

⟨ Process @*n section start 23 ⟩ ≡
```
local match2
match, match2 = string.match(line,"^@%*([0-9]+)%s*([^%s].*)$") if match then
  section["star"] = true
  section["depth"] = tonumber(match)+1
  return section, match2
end
```
This code is used in section 22.

24. ⟨ Process @* section start 24 ⟩ ≡
```
match = string.match(line,"^@%*%*?%s*([^%s].*)$") if match then
  section["star"] = true
  if string_at(line,3) == "*" then section["depth"] = 0 end
  return section, match
end
```
This code is used in section 22.

25. ⟨ Process @␣ section start 25 ⟩ ≡
```
match = string.match(line,"^@%s%s*([^%s].*)$") if match then
  return section, match
end
if string.find(line,"^@%s*$") then return section, nil end
```
This code is used in section 22.

26. The `get_section_name` function is the section name equivalent to `get_section`: if the given line specifies a section name, it returns it. Otherwise, it returns `nil`.

⟨ Parsing functions 16 ⟩ +≡
```
local function get_section_name(line)
  if line and starts_with(line,"@<") then
    local parse, rest = parse_text_line(string.sub(line,3),"@>=")
    if starts_with(rest,"@>=") then return parse["line"] end
  end
end
```

27. Parsing whole sections. We are now ready to read the sections of a CCWEB file. The sections will be stored in the global sections array, which behaves as a map from a section number to the section itself.

⟨ Global variables 27 ⟩ ≡
```
local sections = {}
```
See also section 32.

This code is used in section 5.

28. The read_section function does the main part of the work. It reads the complete definition of the section starting at the given line, adds it to the sections array, and returns the first line that is not a part of the section (and should start another one). The get_next_line parameter should be a function that reads the next line from the input file.

⟨ Parsing functions 16 ⟩ +≡
```
local function read_section(line,get_next_line)
  local section, rest = get_section(line)
  assert(section,"Line '"..line.."' does not start a section.")
  if not rest then line = get_next_line()
  else line = rest end
  ⟨ Read section body 29 ⟩
  ⟨ Read section code 30 ⟩
  table.insert(sections,section)
  return line
end
```

29. We keep adding lines to the section body until we find either the start of another section, or a code chunk to be attached to this section.

⟨ Read section body 29 ⟩ ≡
```
while line and not get_section(line) and not get_section_name(line) do
  table.insert(section["body"],(parse_text_line(line,"$")))
  line = get_next_line()
end
```
This code is used in section 28.

30. Reading the section code is very similar, but we need to skip the current line before adding anything. The resulting loop is exactly the type of loop described by Knuth in "Literate Programming" p.47 as an example of places where goto is justified – in our case, break behaves exactly how goto should.

⟨ Read section code 30 ⟩ ≡
```
section["name"] = get_section_name(line)
if section["name"] then
  while true do
    line = get_next_line()
    if not line or get_section(line) then break end
    local codeline = parse_code_line(line)
    table.insert(section["code"],codeline)
  end
end
```
This code is used in section 28.

31. Parsing entire files. A CCWEB file consists of a *preamble*, which is all that comes before the first section definition, followed by a sequence of section definitions. The preamble will be copied verbatim to the TEX documentation file, at the beginning.

Reading a CCWEB file is a simple matter of initializing a few variables, opening the file for reading, reading the preamble, and calling `read_section` until there is nothing left.

⟨ Parsing functions 16 ⟩ +≡

```
local function read_file(file_name)
  io.stderr:write("(Reading CCWEB file '"..file_name.."')")
  local file, msg = io.open(file_name)
  assert(file,msg)

  get_next_line = file:lines()
  local line = get_next_line()
  ⟨ Read CCWEB file preamble 33 ⟩
  while line do line = read_section(line, get_next_line) end
end
```

32. ⟨ Global variables 27 ⟩ +≡

```
local preamble = {}
```

33. ⟨ Read CCWEB file preamble 33 ⟩ ≡

```
while line and not get_section(line) do
  table.insert(preamble,line)
  line = get_next_line()
end
```

This code is used in section 31.

34. The tangler. The tangler is the program that reads a `ccweb` file and writes the code specified in it. Tangling is done by the `tangle_section` function, which would be rather trivial if it weren't for line breaks and preservation of indentation.

For line breaks, imagine a line like

```
{ @<Something@> }
```

where `@<Something@>` expands to a single line. In this case, we do not want to write any line breaks at either the beginning or the end of `@<Something@>`'s expansion. Indentation follows the same logic, except that we do want to indent the last line – as long as it is not also the first.

The logic is complex to explain, but very easy to implement: we never write anything *after* an expanded line, only *before* it. For the first line, we write nothing before it, while for subsequent lines we write both a line break, and an indentation prefix.

⟨ Tangle file 34 ⟩ ≡

```
local function tangle_section(name,prefix,file)
  local cur_prefix = "" -- Do not write anything before first line
  for n,section in ipairs(sections) do
    if section["name"] == name then
      local ref_prefix, ref_name, pre
      for n,line in ipairs(section["code"]) do
        ⟨ Tangle code line 35 ⟩
        cur_prefix = "\n" .. prefix -- Start indenting and writing line breaks.
      end
    end
  end
end
```

This code is used in section 6.

35. ⟨ Tangle code line 35 ⟩ ≡

```
file:write(cur_prefix)
ref_prefix = prefix
for n,node in ipairs(line) do
  if node then
    if n%2 == 0 then tangle_section(node["line"],ref_prefix,file)
    else ⟨ Tangle code node 36 ⟩
    end
  end
end
```

This code is used in section 34.

36. A code node is simply the code string to be printed.

⟨ Tangle code node 36 ⟩ ≡

```
ref_prefix = ref_prefix .. string.gsub(node,"."," ")
file:write(unquoteline(node))
```

This code is used in section 35.

37. The main routine simply loops through all sections, tangling those that start with "*". The only thing to be careful about is to tangle each section only once.

⟨ Main CCTANGLE procedure 37 ⟩ ≡

```
read_file(arg[1])

local processed_main_sections = {}

for n,section in ipairs(sections) do
  local name = section["name"]
  if name and not processed_main_sections[name] and string.find(name,"^%*") then
    processed_main_sections[name] = true
    io.stderr:write("(Writing main section '"..name.."')")
    if name == "*" then tangle_section(name,"",io.output())
    else
      local file_name, status = string.gsub(name,"^%*{(.*)}","%1")
      assert(status == 1,"Chunk '"..name.."' does not specify a file name.")
      local file = io.open(file_name,"w+")
      assert(file,"Could not open '"..file_name.."' for writing.")
      tangle_section(name,"",file)
      file:close()
    end
  end
end
```

This code is used in section 6.

38. The weaver. The weaver is the program that reads a `ccweb` file and writes out the TEX documentation. This documentation will use `cwebmac` in a way very similar to `CWEB`.

The following table lists some of the macros from `cwebmac.tex` we will use to typeset our documentation. Any macros not mentioned here will be documented as they appear in the code.

Macro	What
`\4`	Backspace one notch.
`\6`	End of named section start.
`\B`	Go into "C mode."
`\M#1`	Begin regular section number `#1`.
`\N#1#2#3.`	Begin starred section `#2`, with title `#3` and depth `#1`.
`\X#1:#2\X`	Begin name definition `#2`, for section `#1`.
`\Y`	Begin a new paragraph before named code.

The TEX preamble and postamble start exactly the same way as in a `CWEB`-produced file:

⟨ TEX preamble 38 ⟩ ≡

```
\input cwebmac
```

See also section 50.

This code is used in section 59.

39. ⟨ TEX postamble 39 ⟩ ≡

```
\inx
\fin
\con
```

This code is used in section 58.

40. The files we will write depend on the "job name," which we initialize with a quick and dirty `basename` replacement. The files written will be *jobname*`.tex`, *jobname*`.idx` (the index), and *jobname*`.scn` (for the section names).

⟨ `CCWEAVE` global variables 40 ⟩ ≡

```
local jobname = string.gsub(string.gsub(arg[1],".*[/\]",""),"%.[^.]*$","")
```

See also sections 41 and 44.

This code is used in section 7.

41. Verbatim print. When CWEB weaves a file, it goes to great lenghts to pretty-print C code. Since CCWEB is language agnostic, it does not even try – it simply typesets all code in \tt font. In order to do that, it must replace all special characters by special sequences:

⟨ CCWEAVE global variables 40 ⟩ +≡

```
tex_quotes = {
  ["&"]  = "{\\AM}", -- Ampersand, from cwebmac
  ["\\"] = "{\\BS}", -- Backslash, from cwebmac
  ["{"]  = "{\\LB}", -- Left brace, from cwebmac
  ["}"]  = "{\\RB}", -- Right brace, from cwebmac
  [" "]  = "~",      -- Non-breakable space
  ["~"]  = "{\\TL}", -- Tilde, from cwebmac
  ["_"]  = "{\\UL}", -- Underline, from cwebmac
  ["^"]  = "{\\CF}", -- Circumflex, from cwebmac
  ["#"]  = "{\\#}",  -- Hash
  ["$"]  = "{\\$}",  -- Dollar
  ["%"]  = "{\\%}",  -- Percent
}
```

42. Quoting a string is simple enough. The \hbox with \tentex inside is a direct copy from cwebmac's \. macro.

⟨ CCWEAVE functions 42 ⟩ ≡

```
local function tex_quoted(line)
  local result = ""
  for c in line:gmatch(".") do
    local repl = tex_quotes[c]
    if repl then result = result .. repl
    else result = result .. c
    end
  end
  return "\\hbox{\\tentex "..result.."}"
end
```

See also sections 45, 48, 49, 51, 52, and 53.

This code is used in section 7.

43. Cross-referencing. Before writing named sections we build a map from their names into which section they were first defined.

⟨ Empty section cross-reference data 43 ⟩ ≡

```
{
  ["secno"]  = -1, -- What is the section number?
  ["more"]   = {}, -- Other sections adding code to this.
  ["usedin"] = {}  -- Sections whose code refer to this.
}
```

This code is used in section 46.

44. ⟨ CCWEAVE global variables 40 ⟩ +≡

```
local cross_ref_map = {}
```

45. We build the map in two passes: the first one fills the "secno" and "more" parts, and the second one fills in the "usedin" list will contain section names.

⟨ CCWEAVE functions 42 ⟩ +≡

```
local function build_cross_ref_map()
  ⟨ Fill in "secno" and "more" data members 46 ⟩
  ⟨ Fill in "usedin" data member 47 ⟩
end
```

46. ⟨ Fill in "secno" and "more" data members 46 ⟩ ≡

```
for n,section in ipairs(sections) do
  local name = section["name"]
  if name then
    local record = cross_ref_map[name]
    if not record then
      record = ⟨ Empty section cross-reference data 43 ⟩
      record["secno"] = n
    else
      table.insert(record["more"],n)
    end
    cross_ref_map[name] = record
  end
end
```

This code is used in section 45.

47. Here we need to make sure we only insert a section once, since one can repeat the reference many times in a single section. Lua does not give us a simple way to do this, but in our case all we need is to check the last value.

⟨ Fill in "usedin" data member 47 ⟩ ≡

```
for n,section in ipairs(sections) do
  for o,codeline in ipairs(section["code"]) do
    for p,node in ipairs(codeline) do if node and p%2 == 0 then
      local name = node["line"]
      assert(cross_ref_map[name],
             "Section @<"..name.."@> does not seem to be defined anywhere.")
      local usedin = cross_ref_map[name]["usedin"]
      if usedin[#usedin] ~= n then
        table.insert(usedin,n)
      end
    end end
  end
end
```

This code is used in section 45.

48. We are going to define the function that writes cross-references here. There's a lot of nitty-gritty logic in here to write just the right macro, but the logic is simple enough.

⟨ CCWEAVE functions 42 ⟩ +≡

```
local function write_cross_reference(prefix,sections,file)
  local n = #sections
  if n == 0 then return end
  file:write(prefix)
  if n > 1  then file:write("s") end
  for k,v in ipairs(sections) do
    if k > 1 then
      if k == n then file:write([[\ET]]) end
      if k == n and n > 2 then file:write("s") end
      if k ~= n then file:write(",") end
    end
    file:write(tostring(v))
  end
  file:write(".")
end
```

49. Writing sections. We start by writing a tex-formatted chunk of documentation. Given a section structure node with `"line"` and `"nodes"` elements, the `write_text_line` function will write it out to the give file.

⟨ CCWEAVE functions 42 ⟩ +≡
```
local function write_text_line(text,file)
  local line = text["line"]
  for n, node in ipairs(text["nodes"]) do
    if node then
      local str = unquoteline(string.sub(line,node[1],node[2]))
      if n%2 == 0 then str = tex_quoted(str) end -- even n: verbatim text
      file:write(str)
    end
  end
end
```

50. To write a section reference, we must look it up in the cross-reference table to find its number. We use a quick and dirty `write_text_line` ∘ `parse_text_line` call to properly parse and format the section name.

⟨ TEX preamble 38 ⟩ +≡
```
\newbox\cctanglebox
```

51. There is one other thing we must take care of: `@<*...@>` sections must be printed verbatim. If the `all_numbers` parameter is true, then write all the section numbers defining this section – this is useful when writing the sections file later on.

⟨ CCWEAVE functions 42 ⟩ +≡
```
local function write_section_reference(name,file,all_numbers)
  local record = cross_ref_map[name]
  assert(record,"Section @<"..name.."@> does not seem to be defined anywhere.")
  file:write("\\setbox\\cctanglebox\\hbox{")
  if string_at(name,1) == "*" then
    file:write(tex_quoted(unquoteline(name)))
  else
    write_text_line((parse_text_line(name,"$")),file)
  end
  file:write([[}\X]])
  file:write(tostring(record["secno"]))
  if all_numbers then for k,v in ipairs(record["more"]) do
    file:write(","..tostring(v))
  end end
  file:write(":\\box\\cctanglebox\\X")
end
```

52. The counterpart to `write_text_line` is `write_code_line`. Given a list with an alternating sequence of code text and section references, it writes it out to the given file.

⟨ CCWEAVE functions 42 ⟩ +≡
```
local function write_code_line(code, file)
  for n, node in ipairs(code) do if node then
    if n%2 == 1 then file:write(tex_quoted(unquoteline(node)))
    else write_section_reference(node["line"],file)
    end
  end end
end
```

53. We are now ready to write the weaved version of an entire section. The `\fi` at the end is necessary to balance out conditionals opened by the `cwebmac` macros issued at every section.

⟨ CCWEAVE functions 42 ⟩ +≡
```
local function write_section(secno,section,file)
  ⟨ Write section start 54 ⟩
  ⟨ Write section body 55 ⟩
  ⟨ Write section code 56 ⟩
  file:write("\\fi\n")
end
```

54. Starred sections get their own table of contents entry, and are specified with a different macro.

⟨ Write section start 54 ⟩ ≡
```
if section["star"] then
  file:write("\\N{"..tostring(section["depth"])..."}{"..tostring(secno)..."}")
else
  file:write("\\M{"..tostring(secno)..."}")
end
```
This code is used in section 53.

55. ⟨ Write section body 55 ⟩ ≡
```
for n,line in ipairs(section["body"]) do
  write_text_line(line,file)
  file:write("\n")
end
```
This code is used in section 53.

56. ⟨ Write section code 56 ⟩ ≡

```
if section["name"] then
  local name = section["name"]
  local record = cross_ref_map[name]

  if table.getn(section["body"]) > 0 then file:write("\\Y") end
  file:write("\\B\\4")
  write_section_reference(name,file)
  file:write("${}")
  if record["secno"] ~= secno then file:write("\\mathrel+") end
  file:write("\\equiv{}$\n")

  for n,line in ipairs(section["code"]) do
    file:write("\\6")
    write_code_line(line,file)
    file:write("\n")
  end

  file:write("\\par")
  ⟨ Write cross-reference information 57 ⟩
end
```

This code is used in section 53.

57. ⟨ Write cross-reference information 57 ⟩ ≡

```
if record["secno"] == secno then
  write_cross_reference([[\A]],record["more"],file)
  write_cross_reference([[\U]],record["usedin"],file)
end
```

This code is used in section 56.

58. The main procedure.

⟨ Main CCWEAVE procedure 58 ⟩ ≡

```
read_file(arg[1])

local file=io.open(jobname .. ".tex","w+")
assert(file,"Could not open '"..jobname..".tex' for writing.")
⟨ Write preamble 59 ⟩
build_cross_ref_map()
⟨ Write index file 60 ⟩
⟨ Write section file 61 ⟩
for n,section in ipairs(sections) do
  write_section(n,section,file)
end

file:write([[⟨ TEX postamble 39 ⟩]])
file:close()
```

This code is used in section 7.

59. ⟨ Write preamble 59 ⟩ ≡

```
file:write([[⟨ TEX preamble 38 ⟩]])
file:write("\n")
for i,line in ipairs(preamble) do file:write(line.."\n") end
```

This code is used in section 58.

60. ⟨ Write index file 60 ⟩ ≡

```
local index_file=io.open(jobname..".idx","w+")
assert(index_file,"Could not open '"..jobname..".idx' for writing.")
index_file:close()
```

This code is used in section 58.

61. The section file has all sections, in alphabetical order. When ordering them, we try, but not too hard.

⟨ Write section file 61 ⟩ ≡

```
local section_file=io.open(jobname..".scn","w+")
assert(section_file,"Could not open '"..jobname..".scn' for writing.")
local sorted_names = {}
for n,section in ipairs(sections) do table.insert(sorted_names,section["name"]) end
table.sort(sorted_names,function (n1,n2)
    return n1:gsub("[^A-Za-z%*]",""):lower() < n2:gsub("[^A-Za-z%*]",""):lower()
  end)
local last_name
for n,name in ipairs(sorted_names) do
  if name ~= last_name then
    local record = cross_ref_map[name]
    section_file:write([[\I]])
    write_section_reference(name,section_file,true)
    write_cross_reference([[\U]],record["usedin"],section_file)
    section_file:write("\n")
    last_name = name
  end
end
section_file:close()
```

This code is used in section 58.

⟨ `*{cctangle.lua}` 6 ⟩
⟨ `*{ccweave.lua}` 7 ⟩
⟨ `CCWEAVE` functions 42, 45, 48, 49, 51, 52, 53 ⟩ Used in section 7.
⟨ `CCWEAVE` global variables 40, 41, 44 ⟩ Used in section 7.
⟨ Common `ccweb` code 5 ⟩ Used in sections 6 and 7.
⟨ Empty body text 14 ⟩ Used in section 17.
⟨ Empty section 13 ⟩ Used in section 22.
⟨ Empty section cross-reference data 43 ⟩ Used in section 46.
⟨ Fill in `"secno"` and `"more"` data members 46 ⟩ Used in section 45.
⟨ Fill in `"usedin"` data member 47 ⟩ Used in section 45.
⟨ Functions 9, 10, 11, 12 ⟩ Used in section 5.
⟨ Global variables 27, 32 ⟩ Used in section 5.
⟨ Insert node in `result["nodes"]` 19 ⟩ Used in sections 18 and 20.
⟨ Main `CCTANGLE` procedure 37 ⟩ Used in section 6.
⟨ Main `CCWEAVE` procedure 58 ⟩ Used in section 7.
⟨ Parse normal text 18 ⟩ Used in section 17.
⟨ Parse verbatim text 20 ⟩ Used in section 17.
⟨ Parsing functions 16, 17, 21, 22, 26, 28, 31 ⟩ Used in section 5.
⟨ Process `@*`n section start 23 ⟩ Used in section 22.
⟨ Process `@*` section start 24 ⟩ Used in section 22.
⟨ Process `@␣` section start 25 ⟩ Used in section 22.
⟨ Read `CCWEB` file preamble 33 ⟩ Used in section 31.
⟨ Read section body 29 ⟩ Used in section 28.
⟨ Read section code 30 ⟩ Used in section 28.
⟨ Tangle code line 35 ⟩ Used in section 34.
⟨ Tangle code node 36 ⟩ Used in section 35.
⟨ Tangle file 34 ⟩ Used in section 6.
⟨ TEX postamble 39 ⟩ Used in section 58.
⟨ TEX preamble 38, 50 ⟩ Used in section 59.
⟨ Write cross-reference information 57 ⟩ Used in section 56.
⟨ Write index file 60 ⟩ Used in section 58.
⟨ Write preamble 59 ⟩ Used in section 58.
⟨ Write section body 55 ⟩ Used in section 53.
⟨ Write section code 56 ⟩ Used in section 53.
⟨ Write section file 61 ⟩ Used in section 58.
⟨ Write section start 54 ⟩ Used in section 53.

CCWEB

	Section	Page
Cesar Crusius' Literate Programming Toolkit	1	1
Main sections	5	2
Utility functions	8	3
Sections	13	5
Basic parsing	16	6
Reading section text	17	7
Reading section code	21	9
Parsing section definitions	22	10
Parsing whole sections	27	11
Parsing entire files	31	12
The tangler	34	13
The weaver	38	15
Verbatim print	41	16
Cross-referencing	43	17
Writing sections	49	19
The main procedure	58	22